MODULE 38 · SECTION 5 - MAINTAINING AND AUTOMATING DATA WORKLOADS

When Pipelines Fail: Restarts, Bad Data, Reprocessing

Idempotent loads, dead-letter topics and tables, quarantining bad records, time travel and snapshots, backfills, and where workers run.

SECTION 5 of 5 EXAM WEIGHT 18% MODULE 38 of 45

admin@sthithapragna.com

Assume every job will run twice. Design it so the second run is harmless.

admin@sthithapragna.com

Five failures, five remedies

WHAT WENT WRONG	REMEDY
Job died **halfway** and reran	Idempotent writes: overwrite or MERGE
A few **bad records**	Dead-letter output, quarantine table
Bad data **already written**	Time travel, or a table snapshot
Data **missing** for some days	Backfill those intervals
A **zone** went down	Regional placement, not one zone

THE PATTERN

Most questions name the failure in one phrase. Match it to the remedy.

admin@sthithapragna.com

Making a rerun harmless

- **Overwrite the partition** for the run date, never append to the table.
- **MERGE** on a business key, so a repeated row updates instead of duplicating.
- Write files to **fixed paths** per run date, so a rerun replaces them.
- **WRITE_APPEND** plus a retry is how duplicates are born.

> THE TRAP
>
> Turning off retries is not idempotency. It just makes failures permanent.

Two safe ways to load the settlement run

```
# 1. Replace only the run date's partition
bq load --replace --source_format=PARQUET \
  'curated.settlement_daily$20261009' \
  'gs://lk-lake-prod/settlement/run_date=2026-10-09/*.parquet'

-- 2. Or upsert on the business key
MERGE curated.settlement_daily t
USING staging.settlement_20261009 s
ON t.run_date = s.run_date AND t.mpan = s.mpan
WHEN MATCHED THEN UPDATE SET kwh = s.kwh, amount_gbp = s.amount_gbp
WHEN NOT MATCHED THEN INSERT ROW
```

2-3 Replace plus a partition decorator: one day only.

9 Match on the key, so repeats update.

THE PAYOFF

Run either one twice. The table ends up exactly the same.

At-least-once or exactly-once?

AT LEAST ONCE

Repeats possible

- Pub/Sub delivery, by design
- STORAGE_API_AT_LEAST_ONCE writes
- At-least-once streaming mode
- Lower latency and cost

EXACTLY ONCE

Repeats removed

- Dataflow dedupes on message ID
- STORAGE_WRITE_API writes
- Exactly-once streaming mode
- Or dedupe yourself on a key

THE TIEBREAKER

A publisher that retries makes a new message ID. Dedupe on an ID attribute, or a business key.

Pub/Sub dead-letter topics

- After **5 to 100** delivery attempts, default 5, a message goes to the dead-letter topic.
- The Pub/Sub **service agent** needs Publisher on that topic and Subscriber on the subscription.
- Pair it with an **exponential backoff** retry policy: 10 to 600 seconds by default.
- The attempt count is **approximate**, and forwarding is best effort.

> THE TRAP
>
> Dataflow reading Pub/Sub? Do not use dead-letter topics or backoff. Dead-letter inside the pipeline.

The dead-letter pattern inside Dataflow

01
Read
From Pub/Sub, retry immediately

›

02
Try to parse
Catch the exception in the DoFn

›

03
Good records
Main output, on to BigQuery

›

04
Bad records
Tagged output, raw payload and error

›

05
Dead-letter table
Fix the code, then replay

REMEMBER

Rows BigQuery rejects come back from getFailedStorageApiInserts. Send them to the same table.

Bad records in batch loads

- **max_bad_records** defaults to 0. One bad row fails the whole load job.
- Raising it lets the job pass, but the bad rows are **silently lost**.
- Better: load raw into **staging**, validate in SQL, move failures to **quarantine**.
- Keep the reason and the source file, then **alert** on the quarantine count.

> THE TIEBREAKER
>
> Lost rows and quarantined rows look the same on a dashboard. Only one can be repaired.

Time travel: undoing a bad change

- Query any table **as it was**, up to 7 days back by default, with FOR SYSTEM_TIME AS OF.
- The window is set per **dataset** or project, from **2 to 7 days**.
- A **deleted** table comes back with bq cp and an @ timestamp in milliseconds.
- **Fail-safe** keeps 7 more days, but only Customer Care can recover it.

> CURRENCY CHECK
>
> Shortening the window saves money only on physical storage billing. On logical billing it is included.

Restoring the tariff history

```
-- A bad MERGE ran at 14:00 yesterday. Rebuild from 13:59.
CREATE OR REPLACE TABLE curated.account_tariff_history_restored AS
SELECT * FROM curated.account_tariff_history
  FOR SYSTEM_TIME AS OF TIMESTAMP '2026-10-09 13:59:00 UTC';

# A table was dropped by mistake? Copy it back from a moment before.
bq cp 'curated.bills@1791385140000' curated.bills_restored
```

4 The table as it stood at 13:59.

7 Epoch milliseconds, inside the window.

THE PATTERN

Restore to a new table first. Check it, then swap it in.

Time travel or a table snapshot?

TIME TRAVEL

Automatic, short

- Always on, nothing to set up
- Any moment in the last 2 to 7 days
- Gone once the window passes
- Covers mistakes you notice quickly

TABLE SNAPSHOT

Deliberate, long

- Read-only copy at one moment
- Kept until it expires, or forever
- Billed only for changed data
- Same region as the base table

THE TRAP

Snapshot before risky changes and at month end. It is not a disaster copy: same region only.

admin@sthithapragna.com

Backfilling the days you missed

- Each run works on its **data interval**, from the logical date, not the clock.
- **Backfill**: Trigger, then Backfill in the UI, or airflow backfill create.
- **Clear** failed tasks to rerun them. Catchup is off by default.
- A streaming gap? Run the **same Beam code** in batch over the archived files.

> REMEMBER
>
> A backfill is only safe if every task is idempotent. It reruns days that may half exist.

Backfilling five nights of settlement

```
# Rerun 1 to 5 October, two nights at a time
airflow backfill create --dag-id settlement_nightly \
  --from-date 2026-10-01 --to-date 2026-10-05 \
  --reprocess-behavior failed --max-active-runs 2

# Or rerun only what failed in one night's run
airflow tasks clear settlement_nightly --only-failed \
  --start-date 2026-10-03 --end-date 2026-10-03
```

3 The intervals to rerun.

4 Only failed runs, two at once.

7 Clearing makes the scheduler run it again.

FOR LARKSPUR

Limit active runs, so the backfill does not starve tonight's real run of slots.

Zones, regions and Dataflow workers

- Set **--region**. With Streaming Engine or Dataflow Shuffle, workers use **every zone**.
- Lose a zone, and workers are **recreated in other zones**. The job carries on.
- **--worker_zone** pins one zone. The docs call it uncommon, and not recommended.
- ZONE_RESOURCE_POOL_EXHAUSTED: that zone has **no capacity**. Let Dataflow choose.

> REMEMBER
>
> Spark clusters are zonal. Keep data in Cloud Storage, so a new cluster can start in another zone.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Retry created **duplicate** rows	Overwrite the partition, or MERGE
Bad messages **stall** the Dataflow job	Catch them, dead-letter table
Push subscriber **keeps failing** on a message	Pub/Sub dead-letter topic
Bad update **yesterday**	Time travel, FOR SYSTEM_TIME AS OF
Keep a copy of month end **for a year**	Table snapshot with expiration
Survive **losing a zone**	--region, no worker zone

THE PATTERN

Repeatable writes, a home for bad records, a way back in time: build all three before you need them.

> An Airflow task loads each night's settlement files into curated.settlement_daily, partitioned by run_date, with WRITE_APPEND.
>
> Last night it failed halfway, Airflow retried it, and that night's totals are now doubled for some meters.

How should the team stop this happening again?

A Turn off Airflow retries, so the task can only ever run once

B Add DISTINCT to every report that reads the settlement table

C Overwrite the run date's partition with WRITE_TRUNCATE

D Raise max_bad_records, so the load finishes on its first attempt

◆ PAUSE AND DECIDE

How should the team stop this happening again?

A No retries

Every transient failure now needs a person.

B DISTINCT in reports

Hides the problem, and costs every query.

C Overwrite the partition

A rerun replaces the day. Retries become safe.

D Raise bad records

Drops rows silently. Duplicates remain.

THE SIGNAL

Duplicates after a retry: make the write idempotent, not the retry rarer.

> After a firmware update, some meters send malformed JSON. The Dataflow parse step throws on them, and the readings job is stalling.
>
> Good readings must keep flowing, and the bad messages must be kept so they can be repaired.

What should the team do?

A Catch the parse error, and write bad messages to a dead-letter table

B Drain the job, then restart it with more workers to clear the backlog

C Filter malformed messages out in the parse step, and discard them

D Add a Pub/Sub dead-letter topic to the subscription, with five attempts

◆ PAUSE AND DECIDE

What should the team do?

A Dead-letter table

Good data flows, bad data is kept for repair.

B Drain, more workers

The same messages fail again on the new job.

C Filter and discard

Flowing again, but the data is lost.

D Dead-letter topic

Not for Dataflow, which acks before parsing.

THE SIGNAL

Dataflow stalls on bad records: dead-letter inside the pipeline.

> A faulty MERGE ran against curated.account_tariff_history at 14:00 yesterday, overwriting thousands of tariff codes.
>
> The last table snapshot is six days old. The table must be put back as it was, losing nothing since.

What should the team do?

A. Ask Customer Care to recover the table from fail-safe storage

B. Restore last week's table snapshot, then rerun six days of loads

C. Re-ingest the whole account history from the change stream

D. Query it FOR SYSTEM_TIME AS OF 13:59 into a new table

◆ PAUSE AND DECIDE

What should the team do?

A Fail-safe

Only after the time travel window has passed.

B Old snapshot

Six days of reloading, and room for new errors.

C Re-ingest

Slow, and the stream may not hold it all.

D Time travel

Yesterday is well inside the window.

THE SIGNAL

A mistake noticed within days: time travel first.

admin@sthithapragna.com

> The readings Dataflow job was launched with --worker_zone=europe-west2-a years ago. A zonal outage stopped it for three hours.
>
> The team wants future jobs to survive losing a zone, with the least ongoing effort.

What should the team do?

A. Run a duplicate copy of the job in europe-west2-b at all times

B. Set only --region, on Streaming Engine, with no worker zone

C. Move the job to a larger machine type in the same zone

D. Take a Dataflow snapshot every hour, so a restart elsewhere is quick

◆ PAUSE AND DECIDE

What should the team do?

A Duplicate job

Twice the cost, and twice the writes to reconcile.

B Region only

Workers spread across zones, and move on failure.

C Bigger machines

Still one zone. The outage stops it again.

D Hourly snapshots

Faster recovery, but still an outage and manual work.

THE SIGNAL

Survive a zone loss: name the region, not a zone.

Module 38 in five lines

- Assume every job runs **twice**. Overwrite the partition, or MERGE on a key.
- Dataflow? Dead-letter **inside the pipeline**. Pub/Sub dead-letter topics are for other subscribers.
- Batch loads: **quarantine** bad rows. Raising max_bad_records loses them.
- **Time travel** for recent mistakes. **Snapshots** for long-lived copies.
- **Backfill** by data interval. Set the **region**, never pin one zone.

COMING UP

Next: module 39, replication, failover and regional outages.